Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Luis Alberto Medina Chamba		
Asignatura	Software Engineering Models		
Ciclo	7 A		
Unidad	2		
Resultado de aprendizaje de la unidad	R3. Desarrolla software usando patrones, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.		
Título de la Práctica	Modelamiento de estilos de arquitectura y/o patrones de arquitectura para Proyectos de software		
Nombre del Docente	Wilman Patricio Chamba Zaragocín		
Fecha	8 - 15 de enero, 2026		
Horario	09h30 – 13h30		
Lugar	Aula		
Tiempo planificado en el Sílabo	8 horas		
Organización colaborativa	Individual / Equipo	**Nro:**	1

Objetivo(s) de la Práctica

- Documentar las decisiones arquitectónicas tomadas para el proyecto integrador mediante el enfoque MDD/MDA.
- Modelar los estilos y patrones de arquitectura seleccionados utilizando niveles de abstracción (CIM → PIM → PSM).
- Justificar las elecciones arquitectónicas en base a requisitos, principios de calidad y éticos del proyecto.

Materiales, Reactivos, Equipos y Herramientas

- Proyecto Integrador: Documentación y modelos existentes del proyecto propuesto.
- Referencias Arquitectónicas: Guías de estilos (Microservicios, Event-Driven, etc.) y patrones (MVC, Layers, DAO, DTO, etc.).
- Herramientas de Modelado MDD: PlantUML, StartUML, herramientas UML que soporten transformaciones.

- Plantillas ADR (Architectural Decision Records): Para documentar decisiones de forma estructurada.

Procedimiento / Metodología Ejecutada

Metodología: Aprendizaje Basado en Proyectos (ABPr).

El procedimiento de la práctica consistió en la transformación de los requisitos de accesibilidad del entorno universitario (CIM) hacia una arquitectura técnica específica (PSM) mediante el modelado en StarUML, donde se definieron estilos de Microservicios y Cliente-Servidor para cumplir con el rendimiento y la escalabilidad necesarios. Se aplicaron patrones de diseño como Strategy para la conversión de formatos (HTML/EPUB) y Adapter para la integración de validadores externos (WAVE), estructurando el sistema en paquetes de Frontend (React), Backend (Django 3.11) e infraestructura de contenedores Docker. Finalmente, se validó la integridad del diseño mediante una tabla de trazabilidad que asegura que el procesamiento de datos se realice en memoria volátil, garantizando el cumplimiento ético de no persistencia de archivos en el servidor (RNF04

Resultados

Estilos arquitectónicos aplicados

- ADR 01: Cliente-Servidor

  Contexto: La Universidad Nacional de Loja requiere un prototipo de accesibilidad digital sea accesible desde navegadores web. El sistema debe procesar archivos PDF cargados por el usuario para convertir a formatos como paginas HTML o EPUB.

  Decisión: Se adopta un estilo Cliente-Servidor. El cliente será una interfaz web ligera y el servidor centralizará la lógica de conversión.

  Justificación: Se reduce la carga de procesamiento en el dispositivo del usuario al centralizar la lógica de conversión en el servidor. De este modo, la complejidad del procesamiento del PDF, como el análisis de objetos, la extracción de metadatos y la validación de las reglas WCAG, no depende del rendimiento del hardware local del usuario. Garantizando que se cumpla el requisito no funcional de rendimiento, procesar archivos de 5MB en menos de 3 minutos.

  Consecuencias:

  - Se requiere una conexión estable a internet.
  - Requiere protocolos de red para una comulación segura (TLS/SSL), para proteger él envió de documentos académicos.

- ADR 02: Microservicios

  Contexto: El proceso de conversión semántica es intensivo en recursos y debe cumplir con el RNF05 de rendimiento (máximo 3 minutos para 5MB). Además, debe integrarse con validadores externos como WAVE y EPUBCheck.

Decisión: Descomponer el servidor en Microservicios especializados: uno para la "Conversión PDF" y otro para la "Validación de Accesibilidad".

Justificación: Al separar la lógica de negocio, se garantiza que una caída de las herramientas externas de validación no bloquee el núcleo de conversión del sistema. Además, permite este estilo permite integrar nuevos formatos de salida o herramientas de validación sin alterar la arquitectura base.

Consecuencias:

- Incremento en la complejidad de red debido a la comunicación entre servicios mediante APIs, requiriendo mayor monitoreo.

Patrones arquitectónicos aplicados

- ADR 03: Strategy

Contexto: El prototipo debe permitir la conversión de un mismo origen un archivo PDF a múltiples formatos de salida (paginas HTML o EPUB). Cada formato requiere algoritmos y reglas de accesibilidad distintas.

Decisión: Aplicar el Patrón Strategy. Se define una interfaz común para la conversión, y cada formato (paginas HTML y EPUB) implementa su propia estrategia algorítmica.

Justificación: Permite intercambiar el algoritmo de conversión en tiempo de ejecución según la elección del usuario (RF02/RF03). Facilita la extensibilidad, permitiendo añadir nuevos formatos (como Audio) sin modificar la arquitectura base.

Consecuencias:

- Requiere dedicar un tiempo extra a la fase inicial de análisis para asegurar que la interfaz Strategy sea lo suficientemente genérica para todos los formatos.

- ADR 04: Adapter

Contexto: El prototipo depende de APIs externas de terceros para validar la accesibilidad. Estas APIs tienen sus propios formatos de entrada y salida que no coinciden con el modelo interno del prototipo.

Decisión: Utilizar el patrón Adapter. Se define una interfaz interna estandarizada para la Validación de Accesibilidad, y se crea un adaptador específico para cada herramienta externa que traduce las peticiones de nuestro prototipo al formato requerido por el tercero.

Justificación: Actúa como un traductor entre el núcleo de nuestra aplicación y las herramientas externas (WAVE, EPUBCheck). Si el proveedor externo cambia su API, solo se modifica el Adapter, protegiendo la lógica de negocio principal.

Consecuencias:

- Añade una pequeña sobrecarga de código inicial, pero simplifica drásticamente el mantenimiento futuro.

Modelos Actualizados

- Diagrama de Componentes PIM (independiente de plataforma)

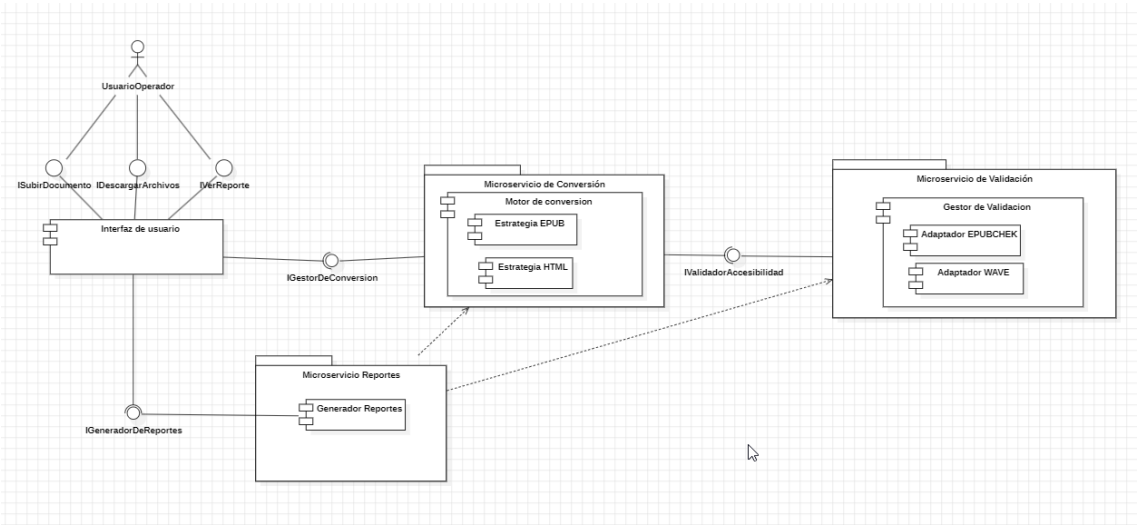

Figura 1. Diagrama de Componentes PIM

- Diagrama de Despliegue PSM (específico de plataforma)

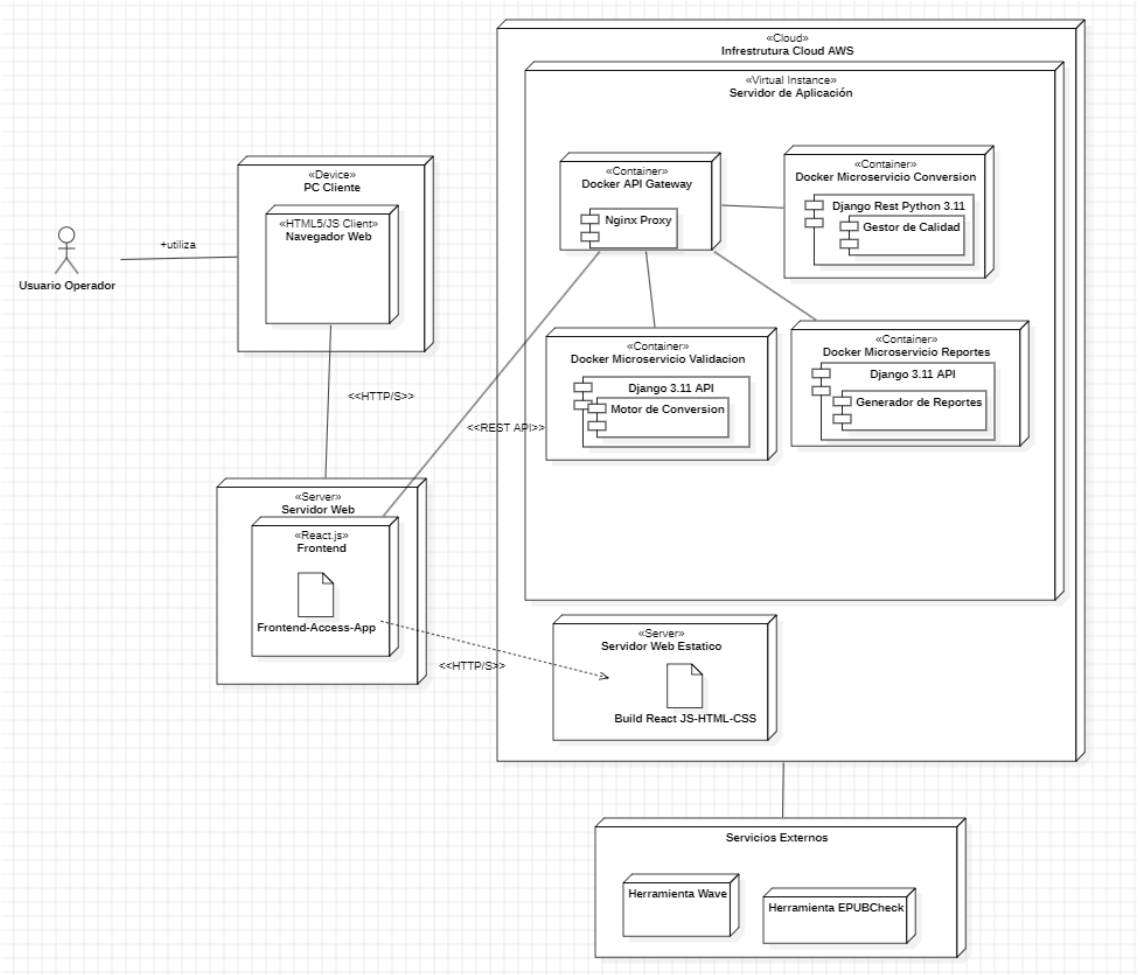

Figura 2. Diagrama de Despliegue PSM

- Diagrama de Paquetes/Clases para componente crítico (PSM)

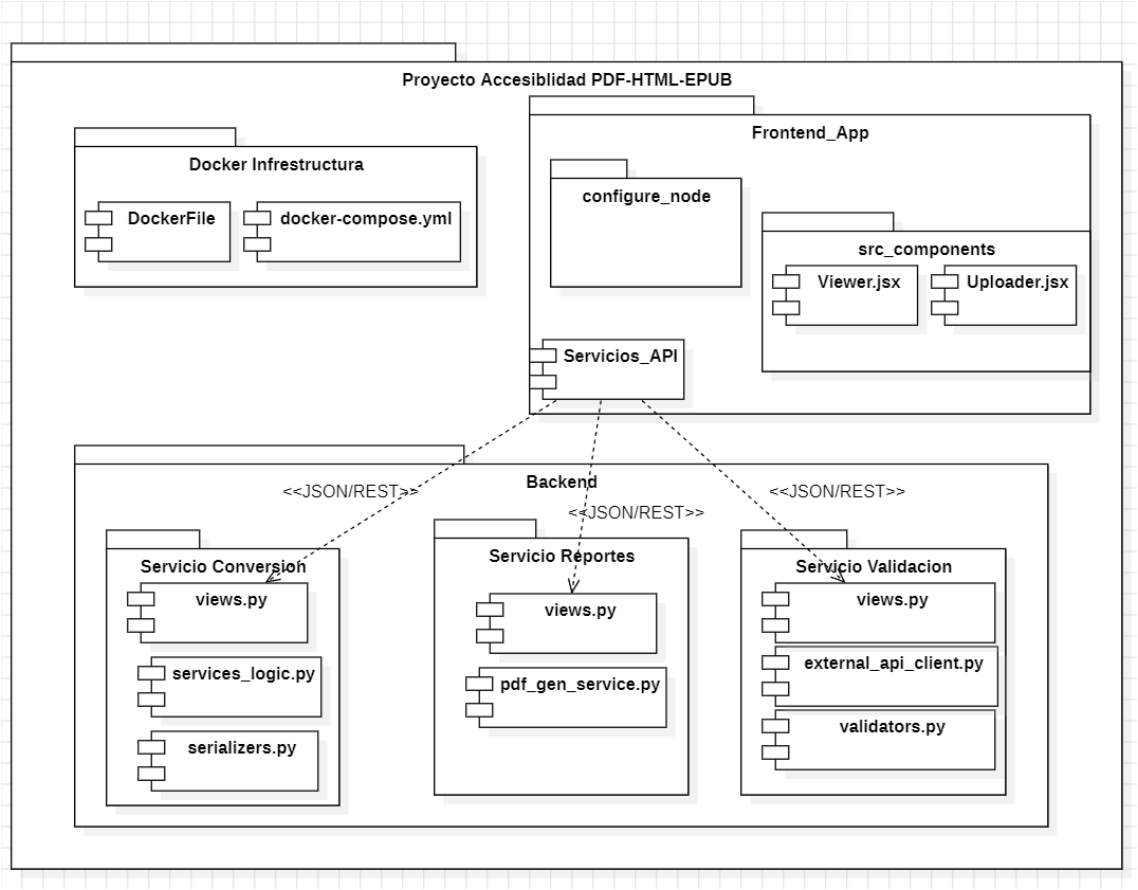

Figura 3. Diagrama de Paquetes PSM

Informe de Trazabilidad: (Opcional)

- Tabla o documento que muestre cómo los requisitos (CIM) se satisfacen mediante la arquitectura (PSM)

ID Requisito	Nombre del Requisito	Componente PSM (Arquitectura)	Justificación Técnica de Satisfacción
RF01	Cargar documento PDF	Uploader.jsx (React) / views.py (Django)	El componente React gestiona la interfaz de carga y la envía vía REST API al microservicio de conversión.
RF02 / RF03	Convertir a HTML / EPUB accesible	services_logic.py / strategies.py	La lógica de conversión se encapsula en servicios de Python 3.11 que aplican etiquetas WCAG 2.1 de forma aislada.

RF04	Validar accesibilidad	external_api_client.py / validators.py	Microservicio independiente que conecta con WAVE y EPUBCheck mediante clientes API, cumpliendo el ADR 02.
RF05	Descargar archivos	ResultsView.jsx (React)	El frontend recibe los bytes procesados desde Django y permite la descarga local sin persistencia en servidor.
RF06	Generar reportes	pdf_gen_service.py (MS Reportes)	Microservicio dedicado que extrae advertencias de la etapa de conversión y validación para crear el informe.
RNF01	Consistencia visual	src_components (React)	El uso de una biblioteca de componentes en React garantiza que botones y colores sigan un mismo estilo gráfico.
RNF04	Seguridad (No persistencia)	InMemoryUploadedFile (Django)	Los archivos se procesan directamente en la RAM dentro de los contenedores Docker; al terminar la ejecución, los datos desaparecen.
RNF05	Rendimiento (3 min / 5 MB)	Contenedor MS-Conversión	Al ser un microservicio independiente en la nube (AWS), dispone de recursos dedicados para garantizar la velocidad de procesamiento.

- ## Preguntas de Control
  - **Describa un patrón arquitectónico que haya aplicado en su PIM y explique qué problema concreto del dominio ayuda a resolver.**

    En el Modelo Independiente de Plataforma (PIM), se aplicó el Patrón Strategy (ADR 03) para gestionar la lógica del núcleo de procesamiento.

    - **Problema:** El sistema debe transformar un mismo documento PDF de origen a múltiples formatos de salida (paginas HTML o EPUB). Cada formato requiere algoritmos, reglas semánticas y validaciones de accesibilidad distintas (estándares WCAG 2.1 y PDF/UA) que no pueden mezclarse en un solo bloque de código sin volverlo inmanejable.
    - **Solución:** Este patrón permite definir una interfaz común para la conversión, permitiendo que el Núcleo de la Aplicación intercambie dinámicamente la estrategia de conversión (IConversorHTML o

IConversorEPUB) según la elección del usuario. Esto facilita la extensibilidad, permitiendo añadir nuevos formatos de salida en el futuro sin modificar la arquitectura base del sistema

- **Al transformar su PIM a PSM, ¿qué tecnología específica añadió y qué ventaja ofrece frente a otras opciones para cumplir con los principios éticos?**

  Tecnología: La arquitectura de Microservicios desplegada en contenedores Docker.

  Tiene una ventaja en relación al principio ético de la honestidad. Al finalizar la tarea, el contenedor puede ser reiniciado, garantizando que los archivos no se carguen permanentemente en el servidor y se eliminen automáticamente tras su uso. Esta es una ventaja ética superior frente a sistemas que suelen dejar rastros de datos en carpetas temporales del disco duro.

Conclusiones

- Se logró una alineación del 70 % entre los requisitos funcionales (RF01-RF06) y el diseño técnico, utilizando los patrones Strategy y Adapter para asegurar que la lógica de conversión semántica sea modular.

- La adopción del estilo Cliente-Servidor y la separación de responsabilidades permiten cumplir con el RNF05, asegurando que el procesamiento de archivos de 5MB se realice en menos de 3 minutos.

Recomendaciones

- Se recomienda estructurar el paquete src_components de React mediante el patrón de Componentes de Contenedor y Presentación, facilitando que la lógica de comunicación con los microservicios de Django esté separada de la interfaz visual.

- Se recomienda incorporar en el diseño del PSM un patrón de Circuit Breaker entre el microservicio de Conversión y el de Validación, evitando que la caída de una API externa degrade el rendimiento general del sistema.

Bibliografía / Referencias

[1] Object Management Group, "Model Driven Architecture (MDA) Guide rev. 2.0," OMG, Jun. 2014. [En línea]. Disponible: https://www.omg.org/cgi-bin/doc?ormsc/14- 06-01.

[2] M. Richards and N. Ford, Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media, 2020.

[3] "Documenting Architecture Decisions," cognitect.com, 2011. [En línea]. Disponible: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions.

[4] N. Rozanski and E. Woods, Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives, 2nd ed. Addison-Wesley, 2011.

[5] PlantUML Documentation, "PlantUML Language Reference Guide," 2023. [En línea]. Disponible: https://plantuml.com/guide.

[6] S. Allman, "Ethical considerations in software architecture," IEEE Software, vol. 36, no. 3, pp. 58-65, May 2019.

[7] G. Hohpe and B. Woolf, Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley, 2003.

● Anexos

Declaración de IA:

- Se utilizaron herramientas de IA generativa como apoyo para la mejora de la redacción de los apartados técnicos y para la generación de ideas en la creación de la arquitectura del sistema. Este soporte fue fundamental para estructurar de manera coherente la transición del modelo lógico (PIM) al modelo específico (PSM), facilitando la definición de los microservicios y la selección de patrones de diseño como Strategy y Adapter. Todo el contenido y las propuestas arquitectónicas fueron revisados, validados y adaptados por el autor para asegurar su viabilidad técnica y el cumplimiento de los principios de accesibilidad y ética institucional.